

Java Swing 图片压缩工具 — 项目实施计划

Context

本项目是南昌航空大学软件工程专业暑期实训项目，目标是开发一款基于 Java Swing 的桌面图片压缩工具。技术栈已确定（JDK 1.8 + Swing + FlatLaf + Thumbnailator + Maven），不可更改。

本计划聚焦于**功能实现顺序**和**模块依赖关系**，将原始规划书转化为可逐日执行的任务清单，确保依赖正确的构建顺序，避免返工。

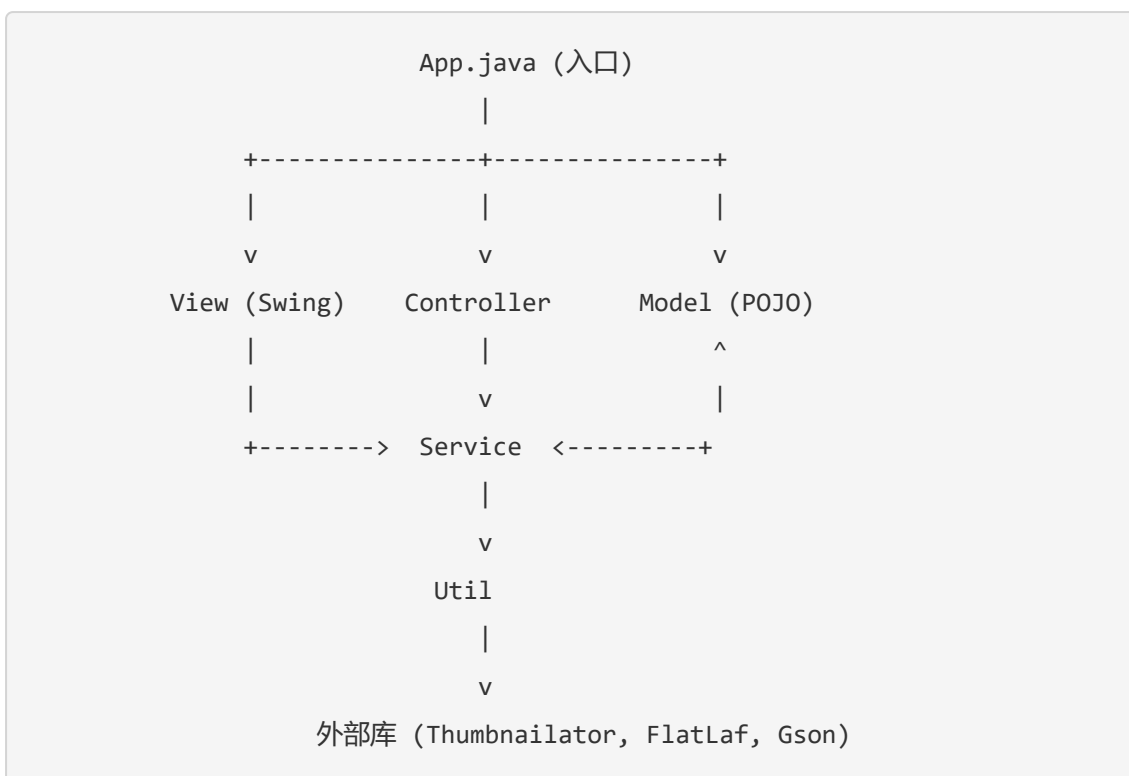
一、改进建议

基于原始规划书，建议以下调整：

#	改进点	原方案	新方案	理由
1	包名	<code>com.compress</code>	<code>com.nchu.imagecompress</code>	遵循反向域名规范，避免与第三方库冲突
2	配置持久化	未明确	JSON 文件 (Gson)，存于 <code>~/.NCHUImageCompressor/config.json</code>	支持嵌套结构，比 Properties 文件更灵活；比 Windows 注册表更可迁移
3	进度回调	未定义接口	<code>ProgressListener</code> 接口 (Service → Controller)	明确的跨层契约，Service 层不引入 Swing 依赖

#	改进点	原方案	新方案	理由
4	UI 构建策略	逐一完整构建面板	骨架先行 ：先搭所有面板的空布局，验证无误后再逐个填充功能	布局问题早期暴露，减少后期大面积返工
5	打包方案	JLink + Launch4j	编译用 JDK 8，打包用 JDK 17 的 jlink 生成精简 JRE	JLink 需要 JDK 9+，JDK 8 不包含此工具
6	易用性优化	保存上次参数	增加 智能质量推荐 （照片/截图/图标三类）、拖拽排序、撤销移除	提升答辩演示的差异化亮点
7	开发周期	14 天笼统划分	14 天按 依赖拓扑排序 精确到半天的任务	每天有明确交付物，可追踪进度

二、模块依赖图（决定构建顺序）



编译依赖规则（箭头 = “依赖于”）：

- **Model**：零依赖，必须最先构建
- **Util**：仅依赖 Model + JDK 标准库，第二批构建
- **Service**：依赖 Model + Util + 外部库，第三批构建（最核心的业务层）
- **Controller**：依赖 Service + Model，第四批构建
- **View**：依赖 Controller + Model，第五批构建
- **App**：组装一切，最后构建

三、跨层关键接口（必须先定义）

3.1 进度回调接口（Service → Controller）

```
// service/ProgressListener.java
public interface ProgressListener {
    void onFileStarted(int index, int total, String fileName);
    void onFileProgress(int index, String fileName, double percent);
    void onFileCompleted(int index, CompressResult result);
    void onBatchCompleted(List<CompressResult> results, int success, int fa
    void onBatchCancelled();
}
```

3.2 视图回调接口 (View → Controller)

```
// controller/MainControllerCallback.java
public interface MainControllerCallback {
    void onImportFiles(List<File> files);
    void onImportFolder(File folder);
    void onRemoveFiles(List<Integer> indices);
    void onClearAllFiles();
    void onStartCompress(CompressConfig config);
    void onCancelCompress();
    void onSelectFile(int index);
    void onToggleTheme();
    void onWindowClosing();
}
```

3.3 核心数据模型

ImageFileInfo	– 文件路径、原始大小、宽高、格式、处理状态、缩略图
CompressConfig	– 质量(0-100)、缩放模式/比例、输出格式、输出路径、命名规则
CompressResult	– 输入信息、输出文件、压缩前后大小、压缩率、成功/失败、耗时
AppConfig	– 主题、窗口位置/大小、上次参数、最近目录 (可序列化为 JSON)

四、分阶段实现计划 (14 天)

第 1 天: 项目骨架 + Model 层

上午: 环境搭建

- [] 创建 Maven 项目, 配置 `pom.xml`
 - 依赖: FlatLaf 3.2.5、Thumbnailator 0.4.20、Gson 2.8.9、JUnit 4.13.2 (test)
 - 插件: `maven-compiler-plugin` (source/target=1.8)、`maven-assembly-plugin` (打包 fat JAR)
- [] 创建完整包结构 `com.nchu.imagecompress`.
`{model,view,controller,service,util}`
- [] 创建 `App.java` 入口类 (空 main 方法, 先验证编译通过)
- [] `mvn compile` 验证环境无误

下午: Model 层 (零依赖, 最先完成)

- [] `ImageFileInfo.java` — 文件信息实体
- [] `CompressConfig.java` — 压缩参数配置

- [] CompressResult.java — 压缩结果
- [] AppConfig.java — 全局配置 (含主题、窗口状态)
- [] ProgressChunk.java — 进度数据块
- [] OutputFormat.java / Theme.java — 枚举类

交付物: 可编译通过的 Maven 项目, 所有数据模型定义完毕。

第 2 天: Util 工具层 + ConfigService

上午: 基础工具类

- [] FileUtil.java — 文件格式校验、扩展名获取、文件大小格式化、目录创建
- [] ImageUtil.java — 读取尺寸、生成缩略图、创建 ImageIcon
- [] ValidateUtil.java — 质量值/缩放比例/输出路径合法性校验

下午: 配置服务

- [] ConfigService.java
 - loadConfig() — 从 ~/.NCHUIImageCompressor/config.json 读取, 缺失则返回默认值
 - saveConfig(AppConfig) — 500ms 防抖写入
 - getDefaultOutputDir() — 返回默认输出目录

交付物: 工具类 + 配置持久化可用, 可编写单元测试。

第 3-4 天: Service 业务层 (核心逻辑)

第 3 天上午: 图片压缩工具类

- [] ImageCompressUtil.java — 封装 Thumbnailator

```
public static CompressResult compress(File input, CompressConfig config,
public static BufferedImage generatePreview(File input, CompressConfig c
```

第 3 天下午: 单张压缩服务

- [] CompressService.java
 - 单张压缩 + 文件命名逻辑 (原名_compressed.ext, 重名自动 _1, _2)
 - 覆盖策略: 默认不覆盖
 - PNG→JPG 透明背景白色填充

第 4 天上午: 批量压缩 + 文件管理服务

- [] ProgressListener.java — 接口定义

- [] BatchCompressService.java
 - 遍历文件列表，逐张调用 CompressService
 - 通过 ProgressListener 回调进度
 - 支持取消（检查 Thread.interrupted() ）
 - 单文件失败不影响整体
- [] FileManagerService.java
 - 导入文件：过滤格式 → 创建 ImageFileInfo → 生成缩略图 → 去重
 - 导入文件夹：递归/非递归扫描

第 4 天下午：单元测试

- [] CompressServiceTest — 质量压缩、尺寸缩放、格式转换、边界值
- [] ConfigServiceTest — 读写往返、缺失文件、损坏 JSON
- [] FileManagerServiceTest — 合法/非法格式、文件夹递归、去重

交付物：完整业务逻辑层，可无界面执行压缩，单元测试通过。

第 5-6 天：UI 骨架（先布局，后功能）

策略：骨架先行 — 先把所有面板的布局搭好，确认窗口、分栏、控件位置正确，再逐个填充交互逻辑。

第 5 天上午：主题工具 + 启动页

- [] ThemeUtil.java — FlatLaf 主题加载、亮/暗切换、全局字体设置
- [] SplashScreen.java — JWindow 启动画面（logo + 进度条 + 版本号）

第 5 天下午：主窗口骨架

- [] MainFrame.java （骨架）
 - JFrame + BorderLayout 四栏布局
 - JMenuBar：文件(导入/导入文件夹/退出)、编辑(清空/设置)、视图(切换主题)、帮助(关于)
 - JToolBar：导入、导入文件夹、清空、压缩、主题切换按钮
 - 中心：JSplitPane(水平) → 左 FileListPanel(36%) | 右 JSplitPane(垂直) → 上 PreviewPanel(60%) | 下 ParamPanel(40%)
 - 底部：StatusBar

第 6 天上午：各面板骨架

- [] FileListPanel.java — JScrollPane + JTable + 底部统计栏 + 空状态插画
- [] PreviewPanel.java — 原图/效果图切换标签 + 画布(JLabel) + 底部数据对比栏

- [] ParamPanel.java — JTabbedPane(基础/高级) + JSlider 质量滑块 + 尺寸设置 + 格式选择 + 操作按钮
- [] StatusBar.java — JLabel 状态文字 + JProgressBar

第 6 天下午：布局验证

- [] App.java 中创建 MainFrame, 加载主题, 显示窗口
- [] 拉伸/缩小窗口验证布局自适应
- [] 切换亮/暗主题验证各面板配色

交付物：完整界面骨架可运行，布局无误，主题可切换。

第 7 天：界面视觉精修

上午：自定义渲染 + 美化

- [] FileListPanel 自定义 TableCellRenderer: 状态图标(待处理/处理中/成功/失败)、奇偶行背景色、文件大小格式化
- [] 全局控件美化: JButton 圆角/悬停色、JSlider 轨道色、JProgressBar 圆角、JScrollPane 细滚动条
- [] FlatLaf 全局默认值定制 (强调色、字体、圆角)

下午：动效 + Toast + 弹窗

- [] AnimationUtil.java — 封装 Timer 动画 (淡入、滑动、脉冲)
- [] ToastNotification.java — 右下角弹出通知 (成功/错误/信息/警告, 3 秒自动消失)
- [] 对话框: 设置对话框、结果通知对话框、确认对话框、错误提示对话框 (统一圆角+阴影风格)

交付物：高保真界面，视觉规范统一，动效流畅。

第 8-9 天：Controller 层 + 功能联调

第 8 天上午：接口 + Worker

- [] MainControllerCallback.java — 接口定义
- [] CompressWorker.java — 继承 SwingWorker<List<CompressResult>, ProgressChunk>
 - doInBackground() → 调用 BatchCompressService
 - process() → EDT 更新进度条和状态文字
 - done() → 处理完成/取消, 显示结果弹窗

第 8 天下午：主控制器

- [] MainController.java

- 构造函数注入所有 Service + MainFrame
- `initialize()` — 加载配置 → 应用主题 → 恢复窗口状态
- 导入流程：选择文件 → FileManagerService → 更新 FileListPanel (EDT)
- 压缩流程：校验 → 禁用按钮 → 创建 CompressWorker → `execute()`
- 预览流程：选中文件 → 加载原图 → 切换效果图预览
- 主题切换：更新配置 → `updateComponentTreeUI` → 保存配置
- 窗口关闭：保存窗口状态 → 保存配置 → 退出

第 9 天：事件绑定 + 完整流程

- [] MainFrame 菜单/工具栏按钮 → Action → Controller 方法
- [] FileListPanel 选择监听 → `Controller.onSelectFile()`
- [] ParamPanel 滑块/输入框监听 → 实时校验 + 保存配置
- [] 拖拽导入：MainFrame.setTransferHandler() → 提取文件列表 → `Controller.onImportFiles()`
- [] 全局快捷键：InputMap/ActionMap 注册 Ctrl+O, Ctrl+S, Delete, Esc 等
- [] 右键菜单：打开原图、打开文件夹、单独压缩、移除、查看详情
- [] `AnimationController.java` — 协调启动页、Toast 队列、按钮动效

交付物：完整可用的单文件压缩流程（导入 → 设参数 → 压缩 → 预览 → 对比），这是项目的里程碑节点。

第 10-11 天：批量处理 + 高级功能

第 10 天

- [] 批量压缩全流程跑通（20 张图片同时处理）
- [] 进度条实时更新 + 当前文件名显示
- [] 中途取消功能（Cancel 按钮 → `worker.cancel(true)`）
- [] 结果弹窗：成功数/失败数/总耗时/节省体积 + 打开输出目录按钮
- [] 失败文件在列表中标红 + 错误信息 Tooltip

第 11 天

- [] 智能参数推荐：根据图片类型（照片/截图/图标）推荐压缩质量
- [] 配置记忆：启动恢复上次参数、主题、窗口大小位置、输出路径
- [] 命名规则：原文件名 / 添加前缀 / 添加后缀
- [] 压缩后自动打开输出文件夹
- [] 列表撤销移除（Ctrl+Z）
- [] 拖拽排序（列表内拖拽调整顺序）
- [] 异常处理完善：文件损坏、路径无权限、磁盘满、超大图片 OOM

交付物：功能完整版本，所有需求功能可用。

第 12 天：细节优化 + Bug 修复

- ☐ 全量回归测试（按测试清单逐项验证）
- ☐ 边界场景：空列表压缩、0 质量、10000px 超大尺寸、损坏文件导入
- ☐ 性能检查：50 张图片批量处理界面不卡顿、内存不泄漏
- ☐ UI 细节：所有分辨率下布局不错乱、控件不溢出
- ☐ 代码规范：包名/类名/方法名统一、核心方法 JavaDoc 注释、复杂逻辑行内注释
- ☐ 清理 System.out.println，改用 Logger

交付物：功能完整优化版，无明显 Bug。

第 13 天：测试 + 打包

上午：测试

- ☐ 功能测试：按测试用例清单逐项验证
- ☐ UI 测试：三种分辨率(1366×768 / 1920×1080 / 2560×1440)、亮暗主题切换
- ☐ 异常测试：非法参数、无权限路径、磁盘空间不足、中途取消
- ☐ 易用性测试：操作步数 ≤3、无基础用户 1 分钟上手

下午：打包

- ☐ Maven Assembly Plugin 配置 → 生成 fat JAR（含所有依赖）
- ☐ JDK 17 jlink → 生成精简 JRE
- ☐ Launch4j 配置 → 生成 Windows .exe（自定义图标、捆绑 JRE）
- ☐ 在无 JDK 环境的电脑上测试运行
- ☐ 整理交付目录结构

交付物：正式稳定版本 + 可执行 JAR + 绿色免安装 EXE。

第 14 天：文档 + 答辩准备

- ☐ 完善实训报告（需求分析、架构设计、核心代码说明、测试报告）
- ☐ 整理项目源码，确保 IDEA 可直接导入运行
- ☐ 制作演示 PPT（突出技术选型、架构设计、界面亮点、功能演示）
- ☐ 准备答辩应答话术（技术选型原因、难点与解决方案、未来扩展方向）
- ☐ 录制操作演示视频（2-3 分钟）

- [] 最终交付物检查清单逐项确认

交付物：源码 + 可运行程序 + 实训报告 + 演示 PPT + 操作视频。

五、每日依赖检查清单

每天开始前确认前置依赖已完成：

天	要开始的任务	前置依赖
1	Model 层	无
2	Util + ConfigService	Model 层完成
3-4	Service 层	Util + Model 完成
5-6	UI 骨架	Service 层可并行进行
7	UI 精修	UI 骨架完成
8-9	Controller + 联调	Service + UI 骨架完成
10-11	批量 + 高级功能	Controller 单文件流程跑通
12	优化修复	全部功能完成
13	测试打包	优化修复完成
14	文档答辩	打包完成

六、技术风险与缓解

风险	严重度	缓解措施
FlatLaf 主题切换导致自定义控件样式错乱	中	第 5 天尽早测试主题切换，优先使用 FlatLaf 标准控件
JLink 需要 JDK 9+，JDK 8 不包含	高	编译用 JDK 8，打包用 JDK 17 的 jlink；或退而求其次只发布 fat JAR
大图片(50MB+)预览OOM	中	预览统一缩放到 1024px 以内；使用 Thumbnailator 流式读取

风险	严重度	缓解措施
批量压缩界面假死	高	严格使用 SwingWorker，所有 UI 更新通过 EDT；进度更新节流(100ms)
文件覆盖丢失原图	高	默认添加后缀，重名自动编号；覆盖选项默认不勾选

七、验证方案

功能验证（核心流程）

1. 启动软件 → 看到启动页 → 进入主界面
2. 拖拽 5 张图片到窗口 → 列表显示 5 条记录（含文件名、大小、尺寸）
3. 调整质量滑块到 60% → 选择 JPG 格式 → 点击压缩
4. 压缩完成 → 预览区自动切换到效果图 → 对比数据显示压缩率
5. 批量导入 20 张 → 一键压缩 → 进度条实时更新 → 结果弹窗显示统计
6. 切换暗色主题 → 所有控件配色同步 → 无样式错乱
7. 关闭软件 → 重新打开 → 上次参数和主题自动恢复
8. 双击 EXE（无 JDK 环境）→ 软件正常运行

异常验证

- 导入损坏图片 → 标记失败，不影响其他文件
- 选择系统保护目录为输出路径 → 提示权限不足
- 批量压缩中点击取消 → 任务终止，已压缩文件保留
- 输入负数质量 → 滑块限制在 0-100，无法输入非法值

八、文件清单（关键产出文件）

```
src/main/java/com/nchu/imagecompress/
├─ App.java                    ← 入口
├─ model/
│   ├─ AppConfig.java         ← 全局配置（JSON 持久化）
│   ├─ CompressConfig.java    ← 压缩参数
│   ├─ CompressResult.java    ← 压缩结果
│   ├─ ImageFileInfo.java     ← 文件信息
│   ├─ OutputFormat.java      ← 枚举
│   ├─ ProgressChunk.java     ← 进度数据
│   └─ Theme.java             ← 枚举
├─ util/
│   ├─ AnimationUtil.java     ← 动效工具
│   ├─ FileUtil.java          ← 文件操作
│   ├─ ImageUtil.java         ← 图片工具
│   ├─ ThemeUtil.java         ← 主题管理
│   └─ ValidateUtil.java      ← 参数校验
├─ service/
│   ├─ BatchCompressService.java ← 批量压缩
│   ├─ CompressService.java    ← 单张压缩
│   ├─ ConfigService.java      ← 配置读写
│   ├─ FileManagerService.java ← 文件管理
│   ├─ ImageCompressUtil.java  ← Thumbnailator 封装
│   └─ ProgressListener.java   ← 进度回调接口
├─ controller/
│   ├─ AnimationController.java ← 动效调度
│   ├─ CompressWorker.java      ← SwingWorker
│   ├─ MainController.java      ← 主控制器
│   └─ MainControllerCallback.java ← 视图回调接口
└─ view/
    ├─ FileListPanel.java      ← 文件列表面板
    ├─ MainFrame.java          ← 主窗口
    ├─ ParamPanel.java         ← 参数设置面板
    ├─ PreviewPanel.java       ← 预览面板
    ├─ SplashScreen.java       ← 启动页
    ├─ StatusBar.java          ← 状态栏
    └─ ToastNotification.java   ← Toast 通知
```